

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитический раздел	4
1.1 Постановка задачи	4
1.2 Описание системы	4
1.3 Общие требования к системе	5
1.4 Требования к функциональным характеристикам	6
1.5 Функциональные требования к системе с точки зрения пользо- вателя	6
1.6 Топология Системы	8
1.7 Требования к программной реализации	9
2 Конструкторский раздел	13
2.1 Сценарии функционирования системы	13
2.2 Диаграммы прецедентов и спецификации сценариев	14
2.3 Спецификация классов	19
2.4 Структурные диаграммы	22
3 Технологический раздел	25
3.1 Средства разработки	25
3.1.1 Выбор СУБД	25
3.1.2 Языки программирования и фреймворки разработки компонентов системы	26
3.2 Пользовательский интерфейс системы	26
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30

ВВЕДЕНИЕ

В 2023 году, по данным Росстат, общее число классифицированных объектов гостиничных услуг составило 31,5 тыс. единиц. С 2022 года данный показатель увеличился на 6,6%, а с 2018 года – на 12,2% [1].

Российский рынок аренды туристического жилья в 2022 году претерпел кардинальные изменения, после того как в начале марта работу в России приостановил наиболее популярный сервис онлайн-бронирования отелей — голландский Booking.com, занимавший более 70% рынка. К ноябрю 2022 года почти 30% бронирований проживания российские пользователи стали совершать напрямую через сайты отелей и других средств размещения, сообщил РБК представитель системы управления гостиницами, отелями, хостелами и апартаментами Bnovo [2].

Анализ рынка дает понять, что бронирование отелей онлайн пользуется большим спросом, особенно ввиду ухода мастодонтов зарубежных сервисов бронирования.

Цель работы – разработать распределенную систему бронирования отелей.

Для достижения цели необходимо решить следующие задачи:

- сформулировать основные требования к системе
- описать требования к подсистемам
- описать архитектуру системы, а так же сценарии её функционирования
- выбрать и обосновать инструменты для разработки программного обеспечения
- реализовать программный продукт.

1 Аналитический раздел

1.1 Постановка задачи

Разработать прототип системы поиска и бронирования отелей на базе web-интерфейса. Система должна состоять из микросервисов, каждый из которых отвечает за свою задачу: сервис пользовательского интерфейса; сервис статистики; сервис авторизации и данных пользовательских аккаунтов; сервис бронирований; сервис оплаты; сервис лояльности; сервис агрегирования запросов и предоставления ограниченного функционала для сторонних приложений. Каждый сервис при необходимости может иметь доступ к связанной с ним базе данных, но не должен иметь доступа к базам данных других сервисов. Все запросы между сервисами требуют авторизацию. Реализовать собственную версию Identity Provider с OpenId Connect. Запросы пользователей делятся на две категории: запросы, требующие авторизации пользователя, и запросы, доступные для всех пользователей, даже неавторизованных. Все ошибки должны обрабатываться; в случае недоступности некритичного функционала должна осуществляться деградация функциональности. Все действия на сервисах должны журналироваться. Информация должна отправляться через Kafka в сервис статистики, где для администратора должна быть реализована возможность построения отчета по пришедшим данным. Все сервисы должны собираться и разворачиваться через CI/CD в kubernetes.

При этом, ПО должно предоставлять отдельные опции пользовательского интерфейса каждому из имеющихся типов пользователей.

1.2 Описание системы

Разрабатываемый сервис должен представлять собой распределённую систему для бронирования номеров гостиниц сети Lux Aeterna Resorts. Если клиент хочет оформить бронь, ему необходимо зарегистрироваться, указав информацию: фамилия, имя, отчество, номер телефона, электронная почта. В случае, если пользователю нужно отменить бронь, получить информацию о бронированиях, статусе в программе лояльности, ему нужно авторизоваться.

На рисунке 1.1 отображена схема предметной области.

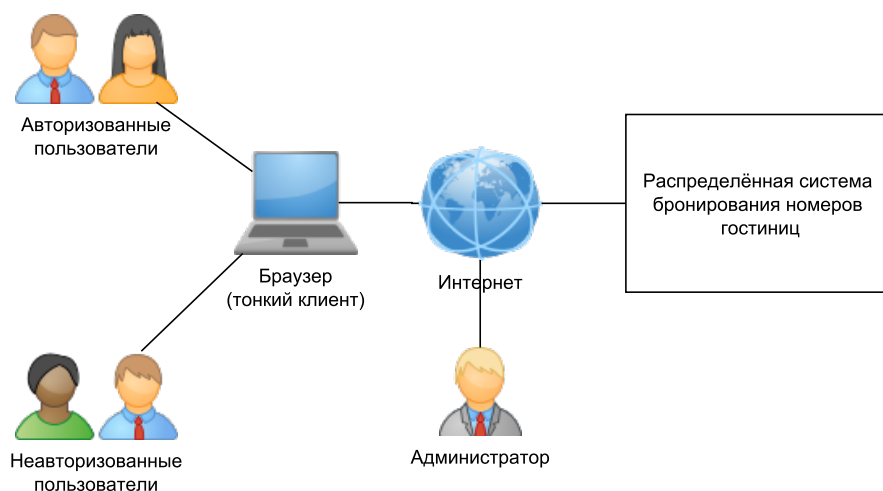


Рисунок 1.1 – Схема предметной области.

1.3 Общие требования к системе

Требования к системе следующие.

- Разрабатываемое ПО должно поддерживать функционирование системы в режиме 24 часов, 7 дней в неделю, 365 дней в году (24/7/365) со среднегодовым временем доступности не менее 99.9%. Допустимое время, в течение которого система недоступна, за год должна составлять $24 \cdot 365 \cdot 0.001 = 8.76$ ч.
- Время восстановления системы после сбоя не должно превышать 15 минут.
- Каждый узел должен автоматически восстанавливаться после сбоя.
- Система должна поддерживать возможность «горячего» переконфигурирования системы. Необходимо предусмотреть поддержку добавления нового узла во время работы системы без рестарта.
- Обеспечить безопасность работоспособности за счёт отказоустойчивости узлов.

1.4 Требования к функциональным характеристикам

- По результатам работы модуля сбора статистики медиана времени отклика системы на запросы пользователя на получение информации не должна превышать 3 секунд.
- По результатам работы модуля сбора статистики медиана времени отклика системы на запросы, добавляющие или изменяющие информацию на портале не должна превышать 7 секунд.
- Медиана времени отклика системы на действия пользователя должна быть менее 0.8 секунд при условии работы на рекомендованной аппаратной конфигурации, задержках между взаимодействующими сервисами менее 0.2 секунды и одновременном числе работающих пользователей менее 100 на каждый сервер, обслуживающий внешний интерфейс.
- Система должна обеспечивать возможность запуска в современных браузерах: не менее 85% пользователей Интернета должны пользоваться ей без какой-либо деградации функционала.

1.5 Функциональные требования к системе с точки зрения пользователя

Система должна обеспечивать реализацию следующих функций.

- Регистрация и авторизация пользователей с валидацией вводимых данных как через интерфейс приложения, так и через социальные сети.
- Аутентификация пользователей.
- Разделение всех пользователей на три роли:
 - Пользователь (неавторизованный пользователь);
 - Клиент (авторизованный пользователь);

- Администратор.
- Предоставление возможностей **Пользователю, Клиенту, Администратору** представленных в таблице 1.1.

Таблица 1.1 – Функции пользователей

Пользователь	1. получение информации об условиях программы лояльности; 2. регистрация в системе; 3. авторизация в системе;
Клиент	1. просмотр списка гостиниц, входящих в сеть; 2. получение информации об условиях программы лояльности; 3. авторизация в системе; 4. просмотр всех бронирований, зарегистрированных на имя текущего клиента; 5. получение детальной информации по конкретному бронированию текущего клиента; 6. бронирование отеля на имя текущего клиента; 7. отмена заказа, оформленного на имя текущего пользователя; 8. получение информации о статусе текущего пользователя в программе лояльности.
Администратор	1. просмотр списка гостиниц, входящих в сеть; 2. просмотр информации о возможности бронирования номера гостиницы по заданным реквизитам; 3. получение информации об условиях программы лояльности; 4. авторизация в системе; 5. получение и редактирование информации о любом клиенте, зарегистрированном в системе; 8. доступ к статистике использования системы.

1.6 Топология Системы

На рисунке 1.2 изображён один из возможных вариантов топологии разрабатываемой распределенной Системы.

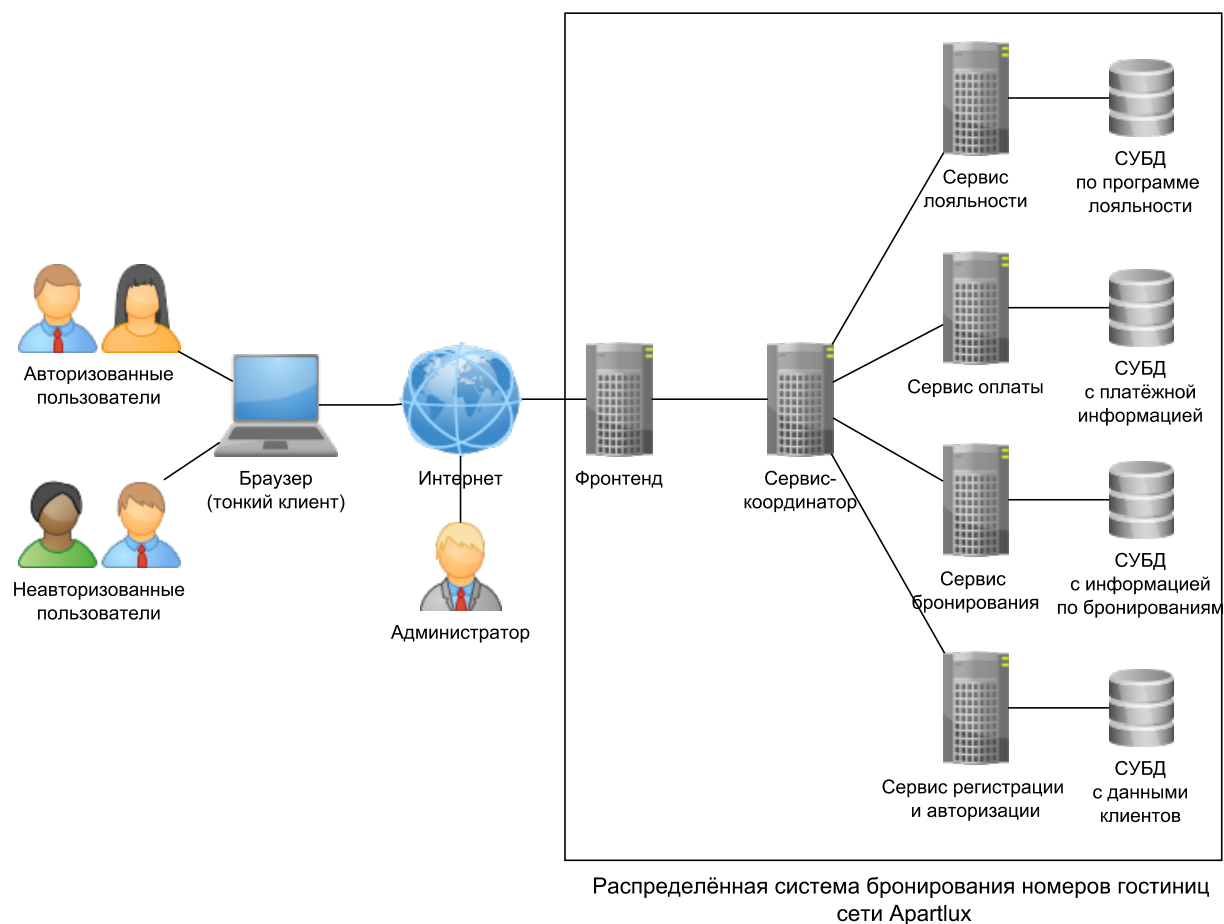


Рисунок 1.2 – Топология системы.

Система будет состоять из фронтенда и 5 подсистем:

- сервис-координатор;
- сервис регистрации и авторизации;
- сервис бронирования;
- сервис оплаты;
- сервис лояльности.

Фронтенд принимает запросы от пользователей по протоколу HTTP и анализирует их. На основе проведенного анализа выполняет запросы к микросервисам бекенда, агрегирует ответы и отправляет их пользователю.

Сервис-координатор – единая точка входа и межсервисной коммуникации.

Сервис-регистрации и авторизации отвечает за:

- возможность регистрации нового клиента;
- аутентификацию пользователя (клиента и администратора);
- авторизацию пользователя;
- выход из сессии.

Сервис бронирования реализует следующие функции:

- получение списка всех гостиниц, входящий в сеть Apartlux;
- получение информации о конкретной гостинице;
- получение, создание, отзыв бронирования.

Сервис оплаты реализует функции:

- проведение платежа от клиента к системе;
- получение статуса оплаты;
- отмену платежа.

Сервис лояльности отвечает за ведение статистики по количеству бронирований всех клиентов, на основе которой для каждого пользователя в индивидуальном порядке предоставляется скидка на будущие заказы.

1.7 Требования к программной реализации

- Требуется использовать СОА (сервис-ориентированную архитектуру) для реализации системы.

- Система состоит из микросервисов. Каждый микросервис отвечает за свою область логики работы приложения и должны быть запущены изолированно друг от друга.
- При необходимости, каждый сервис имеет своё собственное хранилище, запросы между базами запрещены.
- При разработке базы данных необходимо учитывать, что доступ к ней должен осуществляться по протоколу TCP.
- Необходимо реализовать один web-интерфейс для фронтенда. Интерфейс должен быть доступен через тонкий клиент (браузер).
- Для межсервисного взаимодействия использовать HTTP (придерживаться RESTful).
- Выделить Gateway Service как единую точку входа и межсервисной коммуникации. В системе не должно осуществляться горизонтальных запросов.
- При недоступности систем портала должна осуществляться деградация функционала или выдача пользователю сообщения об ошибке.
- Необходимо предусмотреть авторизацию пользователей, как через интерфейс приложения, так и через популярные социальные сети.
- Валидацию входных данных необходимо проводить и на стороне пользователя, и на стороне фронтенда. Микросервисы бекенда не должны валидировать входные данные, поскольку пользователь не может к ним обращаться напрямую, они должны получать уже отфильтрованные входные данные.
- Для запросов, выполняющих обновление данных на нескольких узлах распределенной системы, в случае недоступности одной из систем, необходимо выполнять полный откат транзакции.
- Приложение должно поддерживать возможность горизонтального и вертикального масштабирования за счет увеличения количества функционирующих узлов и совершенствования технологий реализации компонентов и всей архитектуры системы.

- Код хранить на Github, для сборки использовать Github Actions.
- Gateway Service должен запускаться на порту 8080, остальные сервисы запускать на портах 8050, 8060, 8070.
- Каждый сервис должен быть завернут в docker.

Функциональные требования к подсистемам

Подсистемы: фронтенд, бекенд-координатор, бекенд регистрации и авторизации, бекенд бронирования, бекенд оплаты, бекенд лояльности.

Фронтенд – серверное приложение, предоставляет пользовательский интерфейс и внешний API системы, при разработке которого нужно учитывать следующее:

- должен принимать запросы по протоколу HTTP и формировать ответы пользователям в формате HTML;
- в зависимости от типа запроса должен отправлять последовательные запросы в соответствующие микросервисы;
- запросы к микросервисам необходимо осуществлять по протоколу HTTP;
- данные необходимо передавать в формате JSON.

Сервис-координатор – серверное приложение, через которое проходит весь поток запросов и ответов, должен соответствовать следующим требованиям разработки:

- принимать и возвращать данные в формате JSON по протоколу HTTP;
- накапливать статистику запросов, в случае, если система не ответила N раз, то в N + 1 раз вместо запроса сразу отдавать fallback. Через некоторое время выполнить запрос к реальной системе, чтобы проверить её состояние;
- выполнять проверку существования клиента, также регистрацию и аутентификацию пользователей;

- получение списка всех гостиниц с возможными датами для бронирования и внесение изменений в перечень доступных дат (последнее только для администраторов);
- получение информации и обновление данных о зарегистрированном пользователе;
- оформление и отзыв созданного ранее бронирования;
- получение данных о бронированиях пользователя;
- получение статуса конкретного пользователя в программе лояльности и обновление её условий (последнее только для администраторов).

Сервис-регистрации и авторизации должен реализовывать следующие функциональные возможности:

- принимать и возвращать данные в формате JSON по протоколу HTTP;
- возможность регистрации нового клиента и обновление данных уже существующего;
- проверка существования клиента;
- обеспечение авторизации пользователя через аккаунт в системе.

2 Конструкторский раздел

2.1 Сценарии функционирования системы

Регистрация клиента

- Пользователь нажимает на ссылку «Зарегистрироваться» в интерфейсе.
- Пользователь перенаправляется на страницу, которая содержит поля для заполнения его данных.
- Пользователь вводит данные в форму и для завершения регистрации нажимает на кнопку «Зарегистрироваться», тем самым подтверждая верность своих данных, а также согласие на их обработку и хранение.
- Если пользователь с введенным для регистрации логином уже существует, то клиенту возвращается уведомление об ошибке. При успешной регистрации клиент попадает на страницу отелей.

Авторизация клиента

- Пользователь нажимает на ссылку «Войти» в интерфейсе.
- Пользователь перенаправляется на страницу авторизации, которая содержит поля для заполнения логина и пароля.
- Пользователь завершает работу с формой авторизации нажатием кнопки «Авторизоваться».
- При обнаружении ошибки в данных, пользователь получает уведомление об ошибке; при совпадении данных с записью в базе данных аккаунтов пользователь получает доступ к системе.

Бронирование номера

- Клиент нажимает кнопку «Отели».
- Клиент перенаправляется на страницу, которая содержит список гостиниц.
- Клиент нажимает на понравившуюся позицию и попадает на страницу доступных для бронирования номеров в выбранной гостинице с разными реквизитами.
- Клиент нажимает кнопку «Забронировать» напротив нужного номера, на экране появляется всплывающее окно, дублирующее его атрибуты.
- Клиент нажимает кнопку «Подтвердить бронирование», выражая своё согласие на оформление бронирования, и перенаправляется на страницу, где вводит проверочный код из смс. В случае успешной операции придёт смс- и email-оповещения.
- Если клиент не хочет оформлять бронь, он нажимает на кнопку выхода – крестик, всплывающее окно пропадает.

2.2 Диаграммы прецедентов и спецификации сценариев

В системе выделены три роли: Пользователь, Клиент, Администратор. На рисунках 2.1-?? представлены диаграммы прецедентов для каждой из ролей. В таблицах 2.1-2.2 описаны сценарии функционирования наиболее значимых прецедентов.



Рисунок 2.1 – Диаграмма прецедентов с точки зрения пользователя.

Таблица 2.1 – Спецификация сценария регистрации

Нормальный ход сценария	
Действия пользователя	Отклик системы
Регистрация	Система предоставляет пользователю форму для регистрации, в которой нужно заполнить ФИО, дату рождения, логин, пароль, номер телефона, электронную почту
Пользователь заполняет форму и даёт согласие на обработку данных	Данные пользователя регистрируются в системе
Альтернативный ход сценария	
Регистрация	Система предоставляет пользователю форму для регистрации, в которой нужно заполнить ФИО, дату рождения, логин, пароль, номер телефона, электронную почту
Пользователь не заполняет форму или не даёт согласие на обработку данных	Пользователь не заносится в клиентскую базу данных

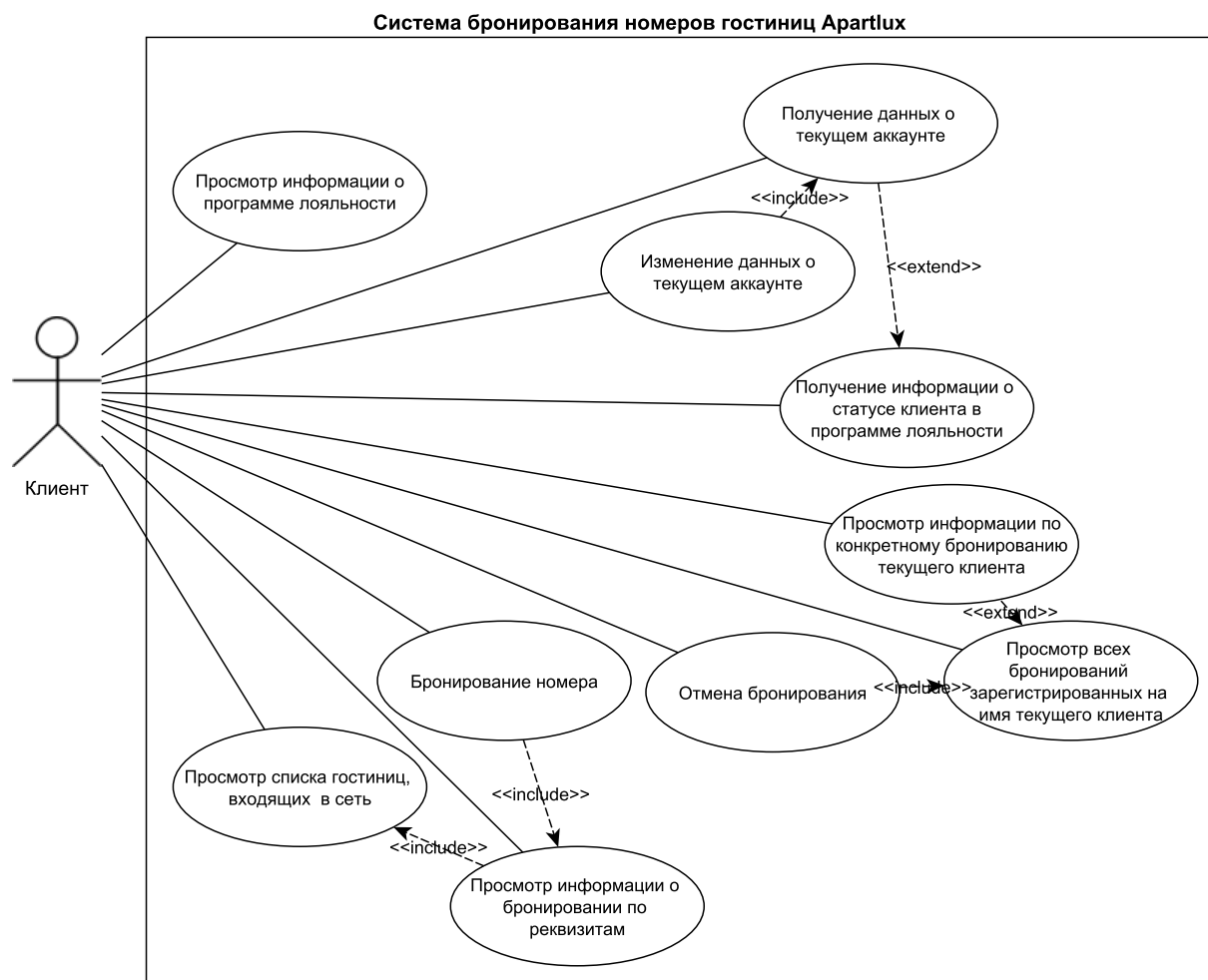


Рисунок 2.2 – Диаграмма прецедентов с точки зрения клиента.

Таблица 2.2 – Спецификация сценария бронирования

Нормальный ход сценария	
Действия клиента	Отклик системы
Просмотр списка гостиниц, предварительно отфильтрованных по выставленным пользователем параметрам	Система предоставляет клиенту список гостиниц, которые входят в состав сети Apartlux и предоставляющих свободные для бронирования номера

Продолжение на следующей странице

Действия клиента	Отклик системы
Просмотр информации о возможном бронировании по заданным реквизитам (дата, цена, количество мест и т.д.)	Система предоставляет клиенту ранжированный список номеров, наиболее подходящих под параметры фильтрации, указанные клиентом
Бронирование	Система перенаправляет клиенту на страницу подтверждения по коду из смс, после успешной операции фиксирует выбранный номер за клиентом
Альтернативный ход сценария	
Просмотр списка гостиниц, предварительно отфильтрованных по выставленным пользователем параметрам	Система предоставляет клиенту список гостиниц, которые входят в состав сети Apartlux и предоставляющих свободные для бронирования номера
Клиент не выбирает ни одну гостиницу из предоставленных	Система отправляет письмо-напоминание на почту клиента, содержащее список предлагаемых гостиниц
Альтернативный ход сценария	
Система не смогла подобрать ни одной гостиницы, которая бы подходила под желания пользователя	Система выводит сообщение об отсутствии позиций, точно подходящих под описание, а также предоставляет список гостиниц, которые частично подходят под реквизиты
Альтернативный ход сценария	
Просмотр списка гостиниц, предварительно отфильтрованных по выставленным пользователем параметрам	Система предоставляет клиенту список гостиниц, которые входят в состав сети Apartlux и предоставляющих свободные для бронирования номера

Продолжение на следующей странице

Действия клиента	Отклик системы
Просмотр информации о возможном бронировании по заданным реквизитам (дата, цена, количество мест и т.д.)	Система предоставляет клиенту ранжированный список номеров, наиболее подходящих под параметры фильтрации, указанные клиентом
Клиент не выбирает ни одного номера и завершает работу с системой	Система отправляет письмо-напоминание на почту клиента, содержащее список предлагаемых номеров
Альтернативный ход сценария	
Просмотр списка гостиниц, предварительно отфильтрованных по выставленным пользователем параметрам	Система предоставляет клиенту список гостиниц, которые входят в состав сети Apartlux и предоставляющих свободные для бронирования номера
Система не смогла подобрать ни одного номера, который бы подходил под желания клиента (дата, цена, количество мест и т.д.)	Система предоставляет клиенту ранжированный список номеров, наиболее подходящих под параметры фильтрации, указанные клиентом
Альтернативный ход сценария	
Просмотр списка гостиниц, входящих в сеть	Система предоставляет клиенту список гостиниц, которые входят в состав сети Apartlux и предоставляющих свободные для бронирования номера
Просмотр информации о возможном бронировании по заданным реквизитам (дата, цена, количество мест и т.д.)	Система предоставляет клиенту ранжированный список номеров, наиболее подходящих под параметры фильтрации, указанные клиентом

Продолжение на следующей странице

Действия клиента	Отклик системы
Бронирование (операции подтверждения по смс завершилась с ошибкой)	Система выводит сообщение об ошибке на экран, из-за проблем с операцией клиенту предлагается повторить попытку, номер на клиента не регистрируется

2.3 Спецификация классов

На рисунке 2.3 представлена диаграмма классов для разработки микро-сервиса бронирования.

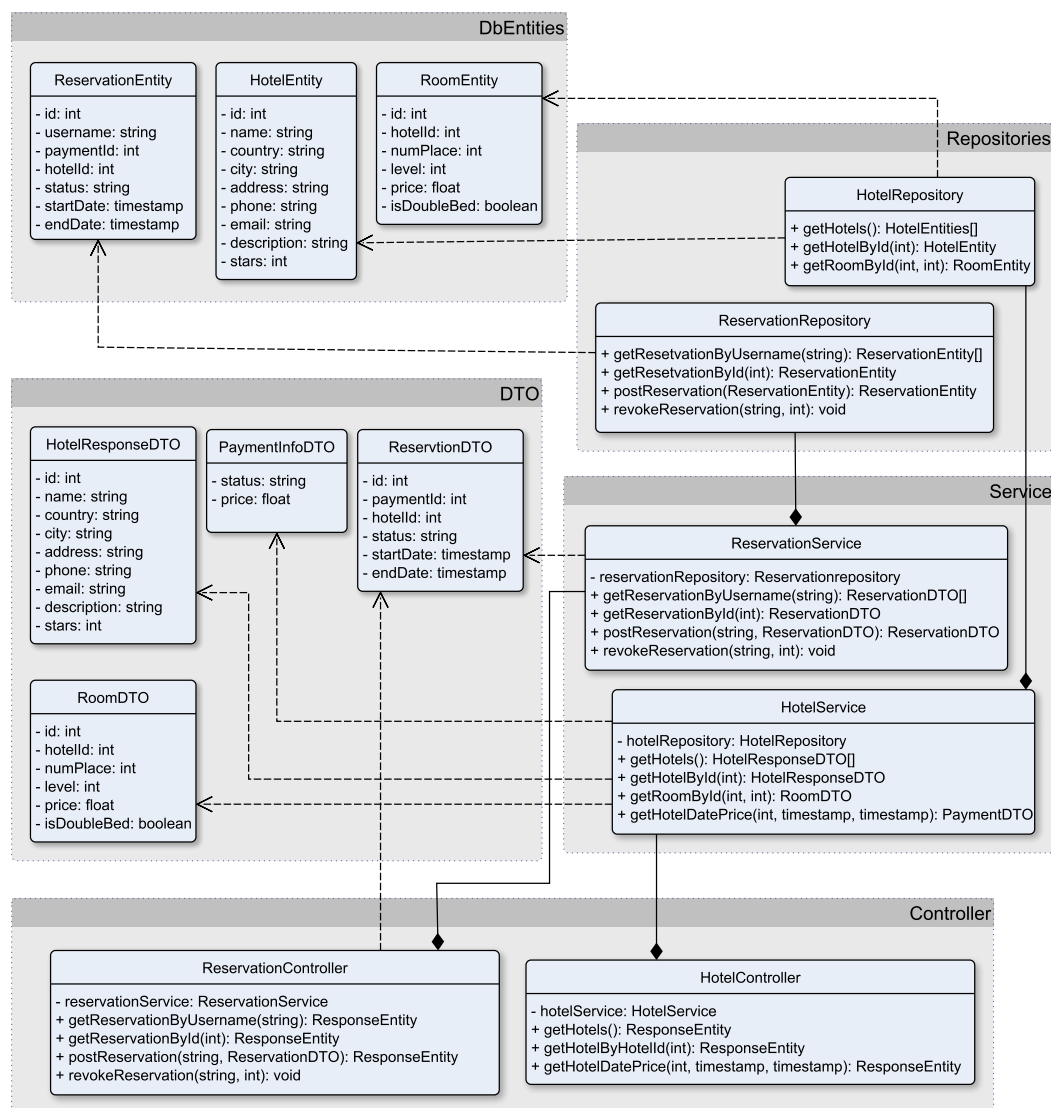


Рисунок 2.3 – Диаграмма классов.

Классы ReservationEntity, HotelEntity представляют легковесные объекты бизнес-логики, ассоциированные с соответствующими сущностями базы данных. Атрибуты указанных классов представлены в таблицах 2.3-2.4.

Таблица 2.3 – Атрибуты класса HotelEntity

Атрибуты		
Имя	Тип	Описание
id	private: int	идентификатор
name	private: string	название
country	private: string	страна
city	private: string	город
address	private: string	адрес
stars	private: int	количество звёзд
price	private: int	цена

Таблица 2.4 – Атрибуты класса ReservationEntity

Атрибуты		
Имя	Тип	Описание
id	private: int	идентификатор
username	private: string	логин клиента
paymentId	private: int	идентификатор платёжной операции
hotelId	private: int	идентификатор гостиницы
status	private: string	статус
startDate	private: timestamp	дата въезда
endDate	private: timestamp	дата выезда

Классы ReservationDTO, HotelResponseDTO, RoomDTO, PaymentInfoDTO представляют объекты для передачи данных между классами бизнес-логики. Атрибутивный состав ReservationDTO, HotelResponseDTO, RoomDTO аналогичен составу ассоциированных с ними сущностей базы данных. Атрибуты вспомогательного класса PaymentInfoDTO представлены в таблице 2.5.

Таблица 2.5 – Атрибуты класса PaymentInfoDTO

Атрибуты		
Имя	Тип	Описание
status	private: string	статус операции
price	private: float	стоимость

Классы HotelRepository и ReservationRepository отвечают за взаимодействие микросервиса с базой данных. Методы каждого приведены в таблицах 2.6-2.7.

Таблица 2.6 – Атрибуты класса HotelRepository

Методы	
Название	Описание
getHotelById(int): HotelEntity	param: id [int - in] - идентификатор <i>получение информации о гостинице по её идентификатору</i>

Таблица 2.7 – Атрибуты класса ReservationRepository

Методы	
Название	Описание
getReservationByUsername(string): ReservationEntity[]	param: [string - in] - логин клиента <i>получение информации о бронированиях по логину клиента</i>
getReservationById(int): ReservationEntity	param: [int - in] - идентификатор бронирования <i>получение информации о бронировании по его идентификатору</i>
postReservation(ReservationEntity): ReservationEntity	param: [ReservationEntity - in] - бронирование <i>создание записи о бронировании</i>

Продолжение на следующей странице

Название	Описание
revokeReservation(string, int): void	param: [string - in] - логин клиента param: [int - in] - идентификатор бронирования <i>отмена бронирования</i>

Классы ReservationService и HotelService реализуют основную бизнес-логику микросервиса, преобразование данных и передачу их на последующий слой, непосредственно связанный с базой данных, поэтому предоставляемые ими методы схожи с методами классов ReservationRepository и HotelRepository.

2.4 Структурные диаграммы

На рисунке 2.4 представлена диаграмма деятельности в режиме «Бронирование» для клиента.

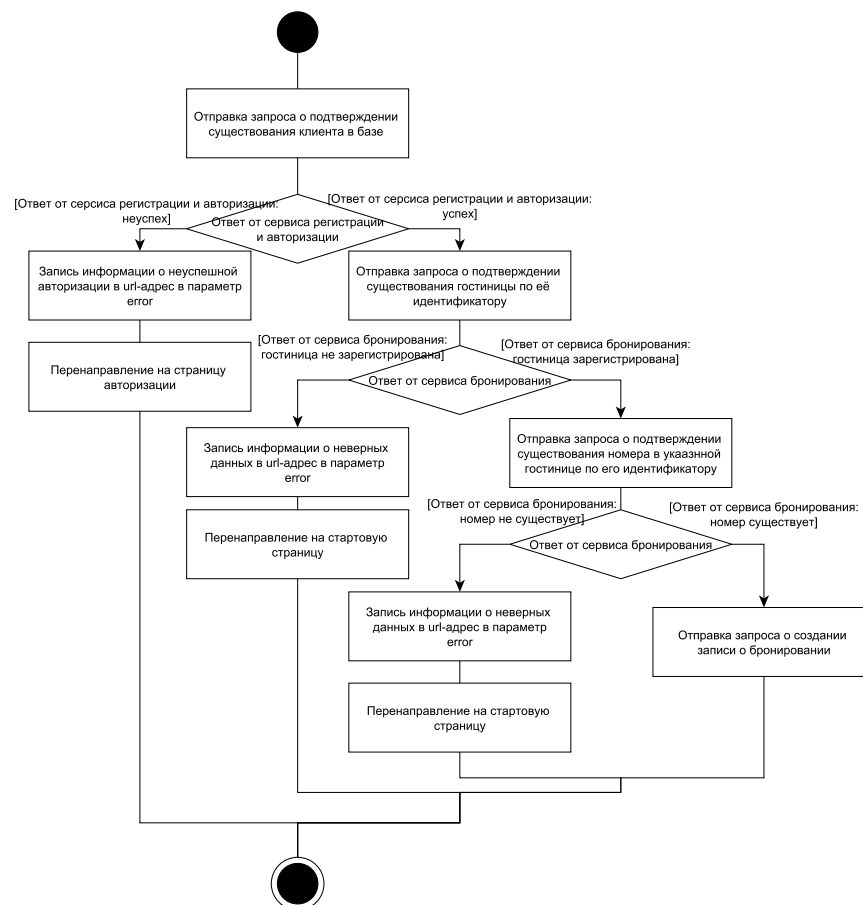


Рисунок 2.4 – Диаграмма деятельности в режиме «Бронирование»

На рисунке 2.5 представлена диаграмма деятельности в режиме «Бронирование».

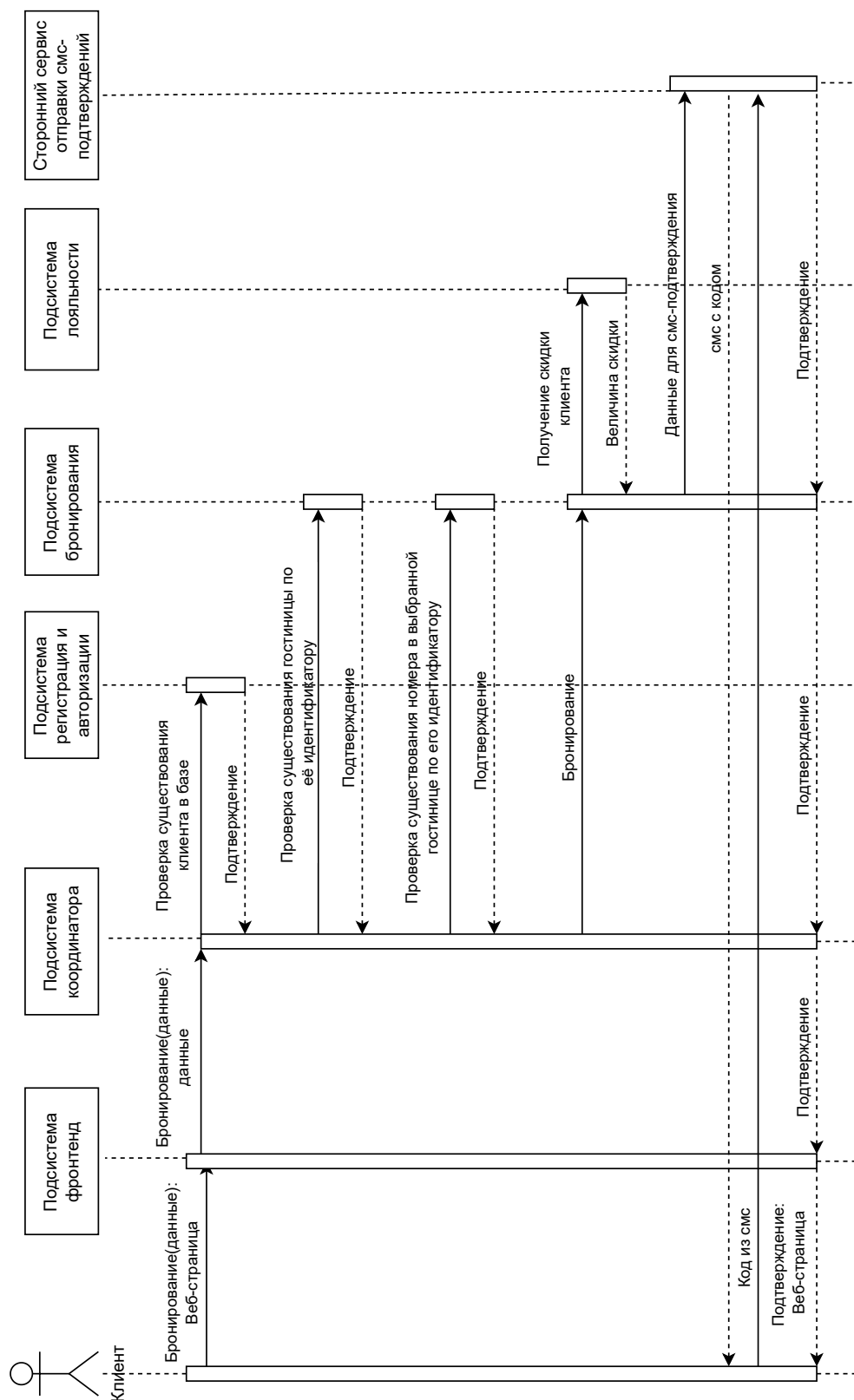


Рисунок 2.5 – Диаграмма последовательности действий при бронировании номера клиентом: концептуальный уровень.

В системе предполагается распределенное хранение данных. Диаграмма потоков данных, представленная на рисунке 2.6.

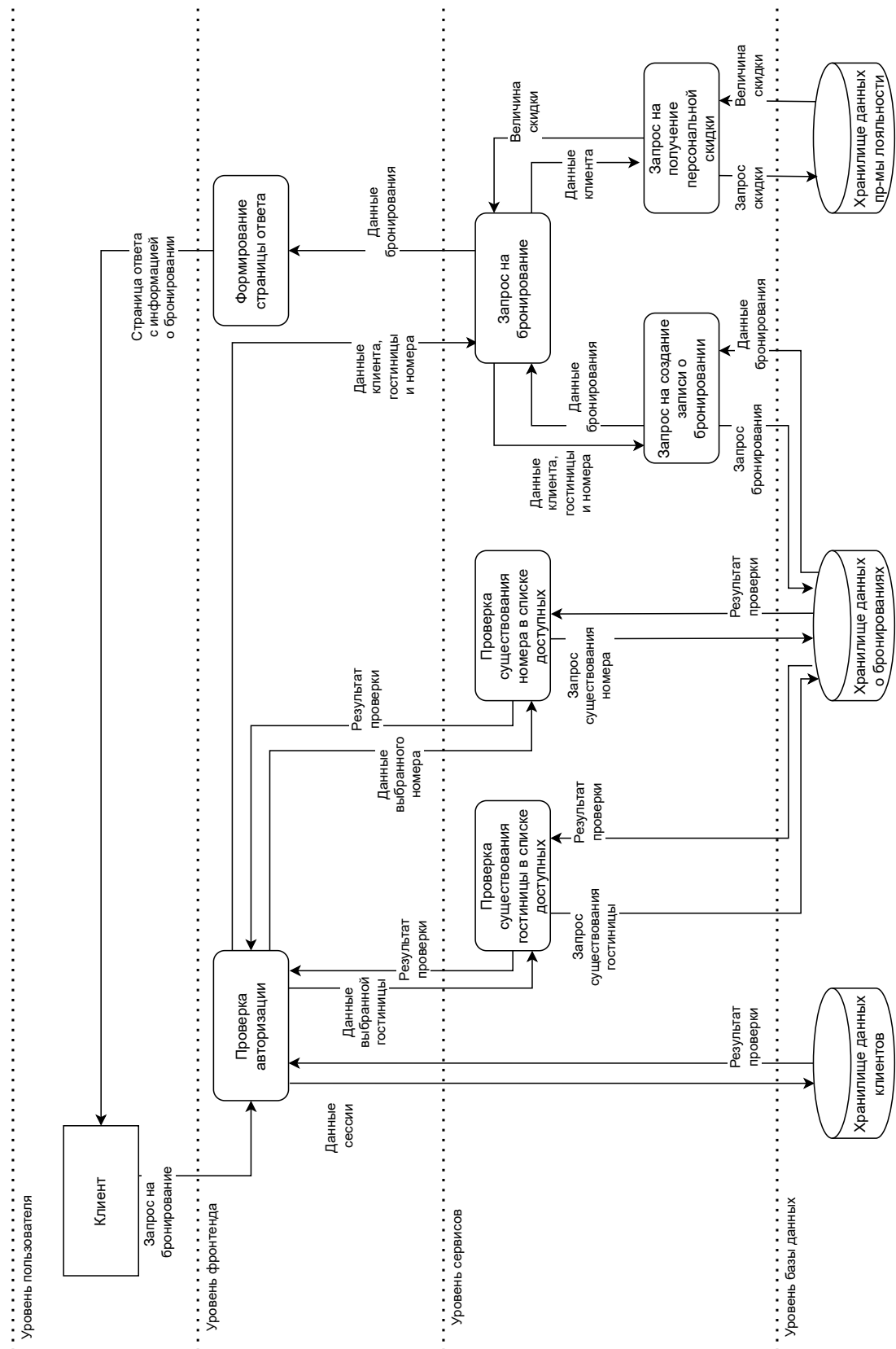


Рисунок 2.6 – Диаграмма потоков данных при бронировании.

3 Технологический раздел

3.1 Средства разработки

3.1.1 Выбор СУБД

В соответствии с техническим заданием разработка бекенда предусматривает следующие требования.

- **Безопасность хранения данных.** Несанкционированный доступ к данным клиентам должен быть невозможен.
- **Транзакционность.** Должен соблюдаться принцип «ACID» (Atomicity — Атомарность, Consistency — Согласованность, Isolation — Изолированность, Durability — Надежность). Атомарность гарантирует, что транзакция не может быть зафиксирована частично. Согласованность — что успешное завершение транзакции оставит систему в согласованном состоянии. Изолированность — что параллельно выполняемые транзакции не будут влиять друг на друга. Надежность — что успешно завершенная транзакция будет зафиксирована, а в случае сбоя, после восстановления системы, результаты транзакции не будут утеряны.
- **Масштабируемость.** Выбранная СУБД должна поддерживать репликацию, шардирование.

PostgreSQL [3] — реляционная система управления базами данных. Она является некоммерческим ПО с открытым исходным кодом. Для работы с этой СУБД существуют библиотеки для таких распространенных языков программирования, как Python, Ruby, Perl, PHP, C, C++, Java, C#, Go. Она работает под управлением многих операционных систем: Linux, MacOS, Windows, FreeBSD, Solaris и др. По сравнению с MySQL система PostgreSQL лучше работает с репликацией, так как в ней существует журнал (средство восстановления системы в случае сбоя) физической модификации страниц. PostgreSQL осуществляет асинхронную репликацию типа «ведущий — ведомый».

Выбор СУБД PostgreSQL для хранения данных разрабатываемой системы обеспечит надежность, безопасность и масштабируемость.

3.1.2 Языки программирования и фреймворки разработки компонентов системы

Средства Backend

- язык программирования – *Python* [4];
- фреймворк *FastAPI* [5] – автоматическая генерация Swagger (простая отладка и тестирование);
- библиотека *OktaJWT* [6] – валидация JWT токенов.

Средства Frontend

- язык – *TypeScript*
- библиотека *React* [8];
- библиотека *Chakra UI* [9] – библиотека компонентов для графического интерфейса.

Каждый сервис и каждый сервер баз данных завернут в отдельный *Docker* контейнер [10].

Вся система завернута в кластер *Kubernetes* [11], арендуемый у *Yandex Cloud*.

3.2 Пользовательский интерфейс системы

Ниже, на рисунках 3.1 – 3.4 представлен пользовательский графический интерфейс системы бронирования отелей Lux Aeterna Resorts.

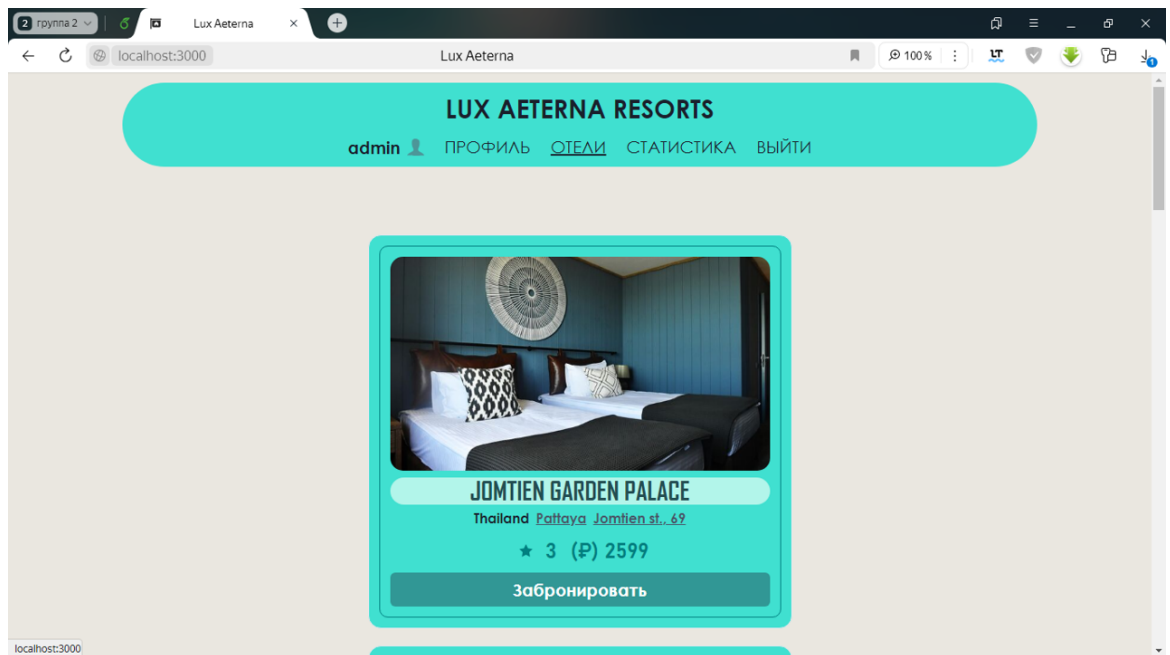


Рисунок 3.1 – Страница отелей

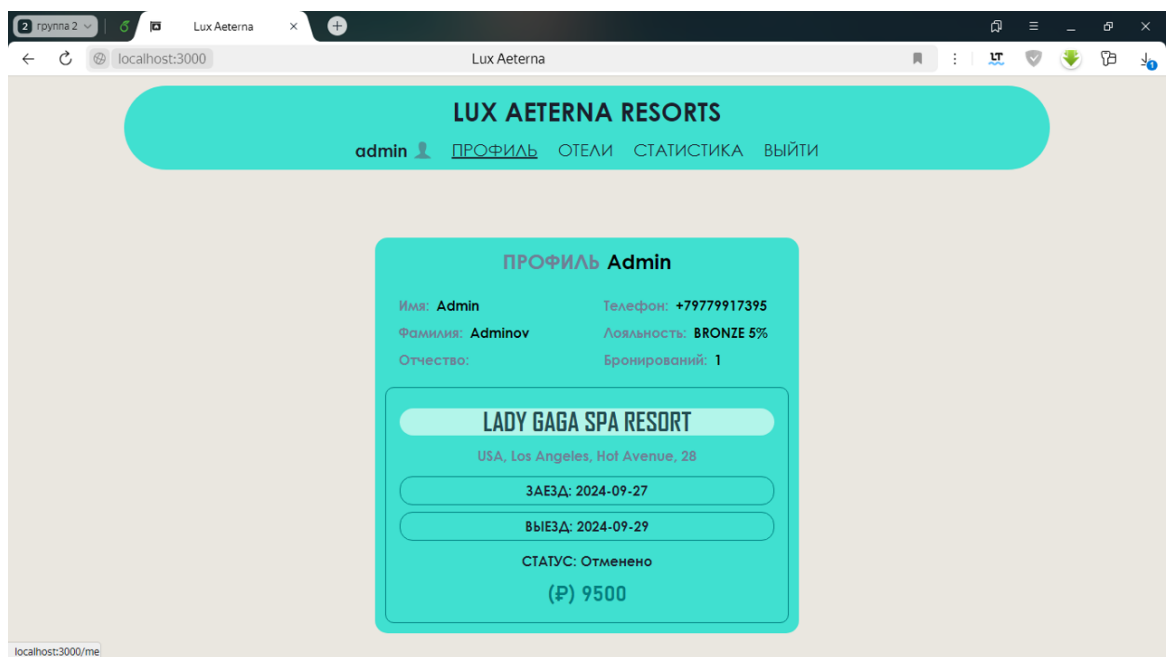


Рисунок 3.2 – Страница пользователя (+ бронирования)

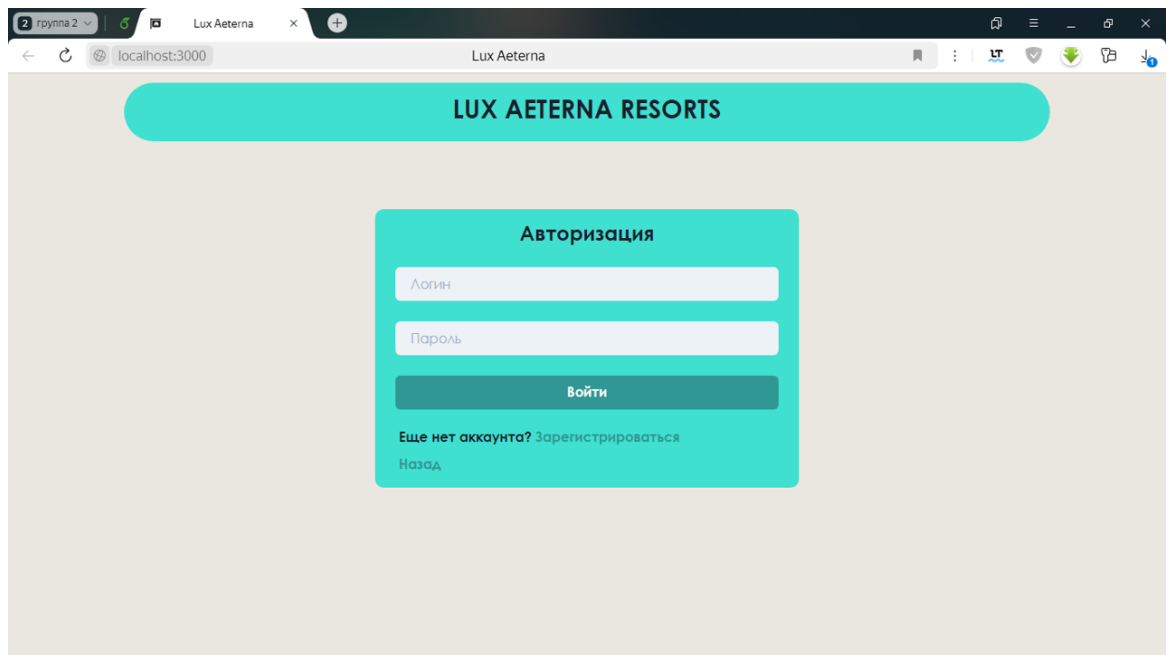


Рисунок 3.3 – Страница авторизации

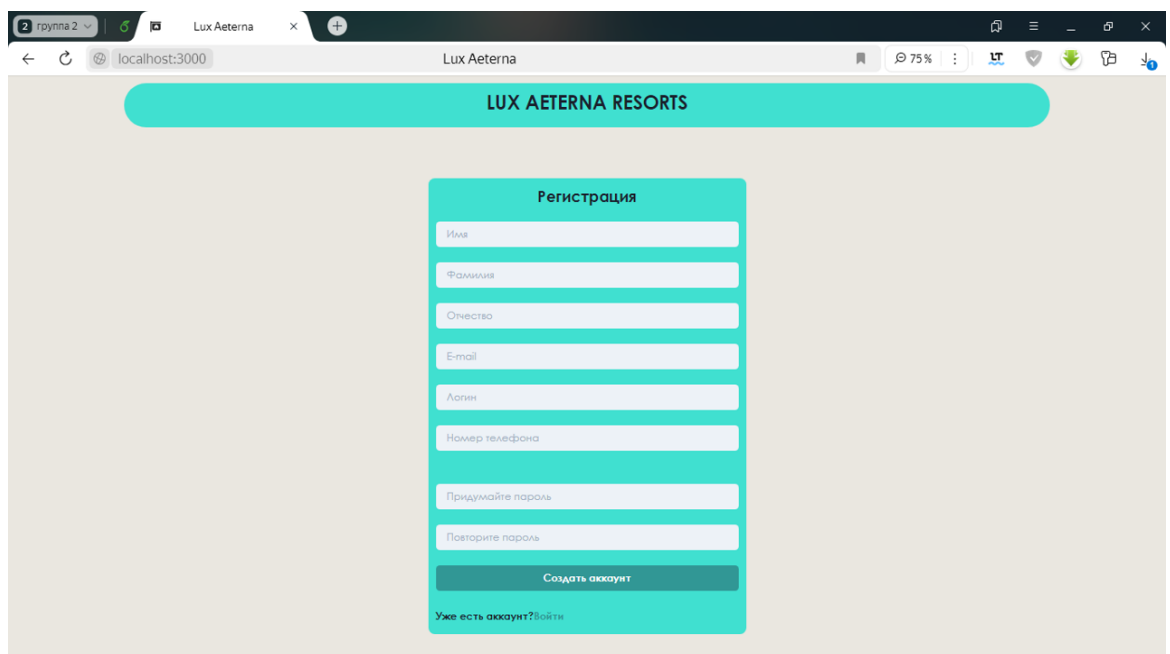


Рисунок 3.4 – Страница регистрации

ЗАКЛЮЧЕНИЕ

В процессе выполнения курсовой работы была разработана распределенная система бронирования отелей сети Lux Aeterna Resorts.

Поставленная цель была достигнута, были выполнены следующие задачи:

- сформулированы основные требования к системе
- описаны требования к подсистемам
- описана архитектура системы, а так же сценарии её функционирования
- выбраны и обоснованы инструменты для разработки программного обеспечения
- реализован программный продукт.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Анализ рынка гостиничных услуг России в 2023 году [Электронный ресурс]. – 2024 – . Режим доступа: <https://research-center.ru/hotels/analysis2023.pdf> (Дата обращения: 10.04.2024).
2. Где россияне бронируют отели после ухода Booking и Airbnb. Инфографика [Электронный ресурс]. – 2022 – . Режим доступа: <https://www.rbc.ru/business/06/12/2022/638dd8fb9a7947d991e9c830> (Дата обращения: 10.04.2024).
3. PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> (Дата обращения: 20.05.2024).
4. Python Documentation [Электронный ресурс]. – Режим доступа: <https://www.python.org/> (Дата обращения: 20.05.2024).
5. FastAPI Documentation [Электронный ресурс]. – Режим доступа: <https://www.python.org/> (Дата обращения: 20.05.2024).
6. OktaJWT Documentation [Электронный ресурс]. – Режим доступа: <https://developer.okta.com/docs/guides/build-self-signed-jwt/java/main/> (Дата обращения: 20.05.2024).
7. TypeScript Documentation [Электронный ресурс]. – Режим доступа: <https://www.typescriptlang.org/> (Дата обращения: 20.05.2024).
8. React Documentation [Электронный ресурс]. – Режим доступа: <https://react.dev/> (Дата обращения: 20.05.2024).
9. Chakra UI Documentation [Электронный ресурс]. – Режим доступа: <https://v2.chakra-ui.com/> (Дата обращения: 20.05.2024).
10. Docker Documentation [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/> (Дата обращения: 20.05.2024).
11. Kubernetes Documentation [Электронный ресурс]. – Режим доступа: <https://kubernetes.io/> (Дата обращения: 20.05.2024).